

Manthan Agent

The No-Backend Skill Pack - 43 skills, full bodies, zero backend, zero OAuth

23

Core skills with full bodies

20

Library skill ports

0

Backends required

0

OAuth clients needed

Constraints: client-side only - browser APIs, Pyodide, CDN libraries and free keyless APIs. No backend server, no OAuth registration, and OCR removed per review. Compiled 25 September 2026 for Manthan Agent v7.9.12 (SahuAI PRO-Lite, Beta).

Scope and constraints

This pack contains every skill Manthan Agent can add under three hard constraints: (1) no backend server - nothing here needs a relay, worker, database or any server-side code; (2) no OAuth - no user-registered clients, no token brokers, no connected-account flows; (3) no OCR - removed per review, so scanned documents without a text layer are explicitly out of scope.

Everything runs on the browser's built-in APIs (Web Speech, Notification, localStorage), the existing Pyodide + esm.sh sandbox, CDN libraries, and free keyless APIs: Open-Meteo, frankfurter.app, Pollinations.ai, Openverse, Nominatim/Overpass, Yahoo/Stooq. GitHub is excluded because it needs a personal access token; video generation, email/calendar, headless browsing and the Meta Ads suite are excluded because they need backends, OAuth or paid models.

The pack has two halves: 23 core skills (new directives, modes and architecture layers) and 20 prompt-only library skills (instruction packs ported into the existing MAKESKILL format).

At a glance

#	Skill	Directive / entry	Tech	Effort
1	chart-studio	CHART: / DIAGRAM: / INFOGRAPHIC:	ECharts + Chart.js via esm.sh; Mermaid.js via esm.sh; matplotlib already in Pyodide	~2 days
2	speak	SPEAK:	Web Speech API speechSynthesis (built into every modern browser)	~0.5 day
3	convert	CONVERT:	Static unit tables in the mcp_calc evaluator + one keyless fetch to frankfurter.app (or open.er-api.com)	~0.5 day
4	weather	WEATHER:	Open-Meteo forecast API + its own geocoding API - free, keyless, CORS-enabled	~0.5 day
5	image-gen	IMG:	image.pollinations.ai - free, keyless, CORS-open; optional OpenRouter image models through existing user keys	~1 day
6	gif-maker	GIF:	Frame generation via image-gen + client-side assembly with gif.js from CDN	~1.5 days (after image-gen)
7	image-search	IMGSEARCH:	Openverse API (free, keyless) + image results from the existing SearXNG instances; Piped already used for YouTube	~0.5 day
8	places	PLACES:	OpenStreetMap Nominatim (geocoding) + Overpass API (POI queries) - both free and CORS-friendly	~1 day
9	stock-snapshot	STOCK:	Yahoo Finance JSON endpoints / Stooq CSV - free and keyless, with a short cache	~0.5 day
10	file-reader	READ_FILE: / FIND_IN_FILES:	pdfjs-dist via esm.sh (or pypdf under Pyodide); SheetJS for xlsx; mammoth.js for docx; Fuse.js (already bundled) for search	~1.5 days
11	pdf-edit	PDFEDIT:	pypdf running under Pyodide on files ingested by file-reader	~1 day (after file-reader)
12	translate	TRANSLATE[from>to]:	Pure LLM shim over the existing chat model	~0.5 day
13	humanizer	HUMANIZE:	Pure LLM shim; rubric from Wikipedia's signs-of-AI-writing guide	~0.5 day
14	audit	AUDIT[type=schema dataset code tos]:	Pure LLM call; dataset and code variants execute real checks in the Pyodide sandbox first	~1 day
15	cite-style	CITE[apa mla chicago ieee harvard]:	Pure prompt with a target-style block	~0.5 day
16	memory	MEMORY: (system-prompt block reading USER_PROFILE)	localStorage - the same store that already persists custom skills	~1 day
17	deep-research	(implicit)	Orchestration over existing SEARCH + READ + Agent Swarm	~1.5 days
18	reminders	REMIND:	Notification API + setInterval; schedule list in localStorage	~0.5 day
19	summarize	SUMMARIZE:	Chains the existing FETCH/READ directives; YouTube transcripts via the Piped API already in use	~0.5 day
20	tdd-mentor	MENTOR: / TDD:	Prompt skill executing inside the existing runpython/runjs sandbox	~0.5 day

#	Skill	Directive / entry	Tech	Effort
21	design-polish	DESIGN[theme=]:	System-prompt branch biasing live_preview output; Mimo's critique/typeset/polish vocabulary as prompt guidance	~0.5 day
22	self-improvement	(implicit)	Markdown log in localStorage alongside memory	~0.5 day
23	heartbeat	(implicit)	setInterval while open; due-check on app open	~1 day
24	+20 library skills	MAKESKILL ports	prompt-only	~0

How to install

- Built-in (first-class): for each core skill, add the four standard pieces to index_2.txt - (i) an entry in the SAHU_SKILLS array (id, name, icon, color, desc); (ii) a system-prompt branch in buildSearchLoopPrompt(); (iii) a directive extractor (regex + handler); (iv) a card renderer for the user-facing bubble.
- Library skills: port each as a built-in custom skill in the existing MAKESKILL format - they appear under Custom skills with their own on/off toggle, persist across reloads, and need no code changes at all.

Part 1 - Core skills (full bodies)

1. chart-studio

CHART: / DIAGRAM: / INFOGRAPHIC: | ECharts + Chart.js via esm.sh; Mermaid.js via esm.sh; matplotlib already in Pyodide | effort: ~2 days

Description. Renders charts, diagrams and infographics as inline interactive artifacts from a JSON or Mermaid specification emitted by the model.

Purpose. The single biggest visible gap in Manthan. Every other document-capable assistant renders charts as first-class output; Manthan currently only embeds static matplotlib images produced inside the code sandbox, which the model must hand-roll every time. chart-studio makes data visualization a directive: the model describes the chart, the app renders it with a real charting engine, and the user gets a consistent, polished, re-editable artifact.

When to use

- Any request for a chart, graph, plot or data visualization with two or more comparable series.
- Flowcharts, architecture diagrams, mind maps, org charts, timelines and Gantt-style views.
- Infographic-style layouts of key metrics (21 preset layouts, in the spirit of baoyu-infographic).
- Supporting visuals requested inside a report or deck that will be produced by File Creation.

How it works in Manthan. The model emits CHART: followed by a fenced JSON block in ECharts option format, or DIAGRAM: followed by Mermaid code. The handler loads ECharts (or Mermaid) from esm.sh inside the existing sandboxed iframe - the same import pattern already used for pptxgenjs and jszip - renders to a canvas, and injects an inline chart card into the chat. INFOGRAPHIC: picks a numbered layout preset and fills it from a key-value spec. For static export, the same directive routes through matplotlib in Pyodide and hands the PNG to the existing FILE[] download card.

Output. An inline interactive chart card with a download-as-PNG button and an 'edit this chart' button that re-prompts the model with the current spec. Mermaid diagrams render as SVG with pan/zoom.

Examples

- CHART: bar - monthly revenue by region, data below
- DIAGRAM: flowchart of the user-onboarding process
- INFOGRAPHIC: 6 startup metrics for a pitch deck, layout 3

Limitations

- Very large datasets should be pre-aggregated in the code interpreter first; the chart spec stays small.
- Mermaid and ECharts versions are pinned via esm.sh for reproducibility.
- Chart interactivity lives in the chat card only; exported PNGs are static.

2. speak

SPEAK: | Web Speech API speechSynthesis (built into every modern browser) | effort: ~0.5 day

Description. Reads any answer or pasted text aloud using the browser's built-in speech synthesis - free, offline, no key, no model call.

Purpose. Manthan has speech-to-text (Whisper) but no text-to-speech, so the voice loop is half-finished. `speak` completes it with the cheapest possible mechanism: the browser already ships a speech engine, so this adds roughly thirty lines and zero dependencies.

When to use

- The user asks to hear an answer, a summary or a passage aloud.
- Accessibility use: reviewing long output while away from the screen.
- Language practice: hearing a translation produced by the `translate` skill.

How it works in Manthan. Two entry points. (1) A speaker icon on every assistant bubble - pure UI, no model round-trip. (2) A `SPEAK` directive for control over what is read: `SPEAK: [rate=0.9] [lang=hi] <text>`. The handler picks the best available `speechSynthesis` voice for the detected language, with rate and pitch flags, and shows a stop control while speaking.

Output. Audio playback inline in the chat. No network calls are made at all.

Examples

- `SPEAK`: the paragraph above, slower
- `SPEAK`: [lang=hi] the Hindi translation

Limitations

- Voice quality depends on the OS and browser (Chrome and Edge are best on Windows; iOS Safari has quirks).
- Works offline; nothing is sent anywhere.
- No voice cloning - that requires server-side models and is excluded by the no-backend constraint.

3. convert

`CONVERT`: | Static unit tables in the `mcp_calc` evaluator + one keyless fetch to `frankfurter.app` (or `open.er-api.com`) | effort: ~0.5 day

Description. Unit, currency and timezone conversion in one directive.

Purpose. Conversion questions are high-frequency and currently answered from model memory, which produces occasional wrong factors. A deterministic converter makes them exact. It extends the pattern Manthan already trusts: units are as static and exact as `CALC`, and currency needs exactly one keyless fetch - the same shape as the existing `SearXNG` public-instance fetches.

When to use

- Any unit conversion: length, mass, volume, temperature, area, speed, data sizes.
- Currency conversion at current exchange rates.
- Timezone conversion (delegates to the existing `DATE` skill, which is already IANA-aware).

How it works in Manthan. The unit table is added to the `mcp_calc` tokenizer/parser so `CONVERT`: 180 lb to kg is evaluated deterministically, never by the model. Currency resolves through one fetch to `frankfurter.app` - free, keyless, CORS-open - with a one-hour `localStorage` cache so repeated questions in a session cost one request. The response is a compact copy-card.

Output. A small card: input value, converted value, the exact factor or rate used, and its timestamp for currency.

Examples

- CONVERT: 180 lb to kg
- CONVERT: 25000 INR to JPY
- CONVERT: 14:30 IST to PST

Limitations

- Currency rates are indicative mid-market rates, cached for an hour - not live trading data.

4. weather

WEATHER: | Open-Meteo forecast API + its own geocoding API - free, keyless, CORS-enabled | effort: ~0.5 day

Description. Current conditions and multi-day forecasts for any named place.

Purpose. Weather is the canonical keyless-API skill: Open-Meteo requires no key, allows browser-origin requests, and ships a matching geocoder, so the whole feature is two fetches. It slots into the MCP-tool pattern occupied by DATE today.

When to use

- Current conditions or a forecast for a city or region.
- Travel planning: 'weather in Manali next weekend'.
- Aggregating weather into a brief composed by heartbeat or deep-research.

How it works in Manthan. WEATHER: <place> geocodes the name with Open-Meteo's geocoding endpoint, fetches the forecast, and renders a compact card: temperature now, rain probability, wind, and a three-day strip. Results are cached briefly. The card fits the existing widget-card styling.

Output. A weather card with current conditions and a short forecast strip, plus a one-line plain-text summary for screen readers.

Examples

- WEATHER: Bhopal
- WEATHER: Tokyo this weekend

Limitations

- Lookup is by place name; optional navigator.geolocation can be offered but is never required.
- No severe-weather push alerts (that would need a background service - excluded).

5. image-gen

IMG: | image.pollinations.ai - free, keyless, CORS-open; optional OpenRouter image models through existing user keys | effort: ~1 day

Description. Text-to-image generation with a transparent-asset flag.

Purpose. Image generation is the headline media capability every full assistant has. It is included here because a free, keyless, browser-callable provider exists (Pollinations), so no account, no key and no backend are needed;

users who already configured OpenRouter keys can optionally route to stronger models through the provider system Manthan already ships.

When to use

- Illustrations, concept art, scenes and icons from a text description.
- Transparent-background PNG assets for decks and UI mockups (the transparent flag).
- Cover images for reports, blog posts and social cards.

How it works in Manthan. IMG: <prompt> issues a direct image fetch to Pollinations with a deterministic seed so re-runs of the same prompt stay consistent. The transparent flag ([transparent]) selects a model that emits alpha-channel PNGs - this covers Meta's transparent-background-image use case. The result lands as an inline image card with a download button.

Output. An inline image card with download-as-PNG and 'regenerate with changes'.

Examples

- IMG: cover illustration for a blog post about deep-sea exploration
- IMG: [transparent] flat icon set of laboratory equipment, minimal style

Limitations

- No true image editing of uploaded photos (provider-side limitation; editing pipelines need keys).
- Free-provider quality is below paid models; content is filtered by the provider.
- Character consistency across separate generations is approximate (seed reuse only).

6. gif-maker

GIF: | Frame generation via image-gen + client-side assembly with gif.js from CDN | effort: ~1.5 days (after image-gen)

Description. Short animated GIFs assembled frame by frame in the browser.

Purpose. Covers Meta's gif-generation skill without any encoder backend: the model plans a small set of frames as a coherent sequence, the app fetches each frame from the keyless image provider, and gif.js (a pure-JavaScript GIF encoder loadable from CDN) assembles them.

When to use

- Simple looping animations, loading indicators, physics or process motion.
- Fun visual accents for chats and decks.

How it works in Manthan. The model expands GIF: <description> into a numbered frame plan - each frame is a full image prompt differing only in the described motion state, plus a shared style block for consistency. The app fetches frames, assembles at the chosen frame rate with gif.js, and shows a preview card with download.

Output. An animated GIF card with playback preview and a download button.

Examples

- GIF: bouncing ball loop, 8 frames, 12 fps
- GIF: sunrise over mountains, 6 frames

Limitations

- Frame-to-frame consistency is imperfect with free generation; descriptions must over-specify fixed elements.
- Six to ten frames is the practical ceiling; longer animations belong in video (excluded - paid server models).
- Depends on image-gen landing first.

7. image-search

IMGSEARCH: | Openverse API (free, keyless) + image results from the existing SearXNG instances; Piped already used for YouTube | effort: ~0.5 day

Description. Finds reference and illustrative images on the open web, with license metadata.

Purpose. Many answers improve dramatically with a visual reference - design styles, art movements, identifiable objects. Openverse gives clean, license-labeled results with no key; the SearXNG instances Manthan already queries serve general web image results as a second source.

When to use

- Reference images for design, art or styling questions.
- Illustrating a point in an answer where seeing beats describing.
- Finding openly licensed images safe to reuse.

How it works in Manthan. IMGSEARCH: <query> queries Openverse first (it returns CC-licensed works with clear attribution metadata) and the image channel of the enabled SearXNG engines as a fallback. Results render as a horizontal carousel card; each image links to its source page and shows its license.

Output. A carousel card of image thumbnails with source links and license labels.

Examples

- IMGSEARCH: art-deco poster design references
- IMGSEARCH: cross-section of a steam turbine, labeled

Limitations

- No reverse image search (upload an image to find its origin).

8. places

PLACES: | OpenStreetMap Nominatim (geocoding) + Overpass API (POI queries) - both free and CORS-friendly | effort: ~1 day

Description. Local search for restaurants, cafes, parks, ATMs, chemists and other services near a place.

Purpose. This is Meta's 'local' skill made keyless: instead of Google Places, it uses the OpenStreetMap stack, which needs no key, no account and no backend. Coverage in India is strong in cities and varies by region - the honest trade-off for zero credentials.

When to use

- 'Find vegetarian restaurants near Connaught Place'.
- Locating practical amenities: ATMs, pharmacies, fuel, parking.
- Building a shortlist for a trip or an evening plan.

How it works in Manthan. PLACES: <what> near <where> geocodes the place with Nominatim, then runs an Overpass query using the matching amenity tags (cuisine, shop, amenity) inside a radius. Results render as a list card with name, distance, opening hours where tagged, and OSM links. An optional static OSM tile map can sit above the list.

Output. A places card: map strip (optional) plus ranked list with distances and tagged details.

Examples

- PLACES: chemists near Saket, Delhi
- PLACES: cafes with wifi near MG Road, Pune

Limitations

- OSM data quality varies by region; rural coverage is thinner.
- No live availability, reservations or reviews - those need keyed APIs (excluded).
- Google Places remains the higher-quality paid alternative if keys are ever allowed.

9. stock-snapshot

STOCK: | Yahoo Finance JSON endpoints / Stooq CSV - free and keyless, with a short cache | effort: ~0.5 day

Description. Quick quotes and lightweight financial snapshots for stocks, indices and currencies.

Purpose. Financial questions are frequent but currently answered from model memory, which goes stale immediately. A snapshot directive gives real delayed quotes without the server-side finance libraries (akshare, Tushare) that are excluded by the no-backend constraint.

When to use

- 'What is Apple trading at?'
- Index and currency levels for a brief.
- Context for finance-flavored research and reports.

How it works in Manthan. STOCK: <symbol> fetches the quote from Yahoo Finance's JSON endpoint (or Stooq CSV as fallback), caches for five minutes, and renders a compact table card: last price, change, percent, day range, 52-week range. Multiple symbols run in parallel, one per line.

Output. A quote card per symbol with a timestamp and data-source line.

Examples

- STOCK: AAPL
- STOCK: ^NSEI ^BSESN USDINR

Limitations

- Quotes are delayed, not live trading data.
- A-share and some regional data need the excluded server libraries.
- Informational only - never investment advice, and the card says so.

10. file-reader

READ_FILE: / FIND_IN_FILES: | pdfjs-dist via esm.sh (or pypdf under Pyodide); SheetJS for xlsx; mammoth.js for docx; Fuse.js (already bundled) for search | effort: ~1.5 days

Description. Uploads documents and reads them into the conversation: a router that sends each file type to the right parser, plus search across everything attached.

Purpose. File Creation is output-only today; this is its mirror image. Once users can attach a PDF, spreadsheet or document and have the model actually read it, a whole class of tasks (analyze this invoice, extract this table, what does this contract say) becomes native. It is also the prerequisite for pdf-edit and unlocks the dataset skills.

When to use

- The user uploads a PDF, DOCX, XLSX, CSV, JSON or plain-text file and asks about its contents.
- Comparing or merging information across several uploaded files.
- Searching for a phrase across all attachments of a session.

How it works in Manthan. An upload button (the app already accepts files for vision models) routes each file by extension to the right parser inside the sandbox: pdfjs-dist for PDF text and page structure, SheetJS for workbook sheets, mammoth.js for Word documents, native parsing for text formats. READ_FILE: <name> [page/sheet] feeds the extracted content back into the conversation exactly like a READ: result. FIND_IN_FILES: <query> indexes every attached file with Fuse.js and returns matched excerpts with file and location.

Output. A quiet 'loaded' card per file (type, pages/rows, size), then the model's answer drawing on the real content; FIND_IN_FILES returns an excerpt list.

Examples

- READ_FILE: invoice.pdf
- READ_FILE: budget.xlsx sheet 2
- FIND_IN_FILES: refund policy

Limitations

- Scanned PDFs with no text layer return nothing - OCR has been explicitly excluded from this pack.
- Very large files are truncated with a clear notice of what was cut.
- Images and audio route to the existing vision/Whisper paths, unchanged.

11. pdf-edit

PDFEDIT: | pypdf running under Pyodide on files ingested by file-reader | effort: ~1 day (after file-reader)

Description. Merges, splits, rotates, reorders, watermarks and edits metadata of uploaded PDFs - fully client-side.

Purpose. This is the feasible half of advanced PDF work: everything pypdf can do runs fine in Pyodide, so merge/split/rotate/watermark need no server. The infeasible half (OCR on scans, heavy layout surgery) is deliberately out of scope.

When to use

- Combining several PDFs into one.

- Extracting a page range; rotating scanned pages.
- Stamping a DRAFT or CONFIDENTIAL watermark before sharing.

How it works in Manthan. PDFEDIT: <operation> operates on files already loaded by file-reader, executes pypdf in the Pyodide sandbox, and hands the result to the existing FILE[] card as a download. Supported operations: merge, split/extract pages, rotate, reorder, watermark (text, opacity, position), and metadata edit.

Output. A download card for the edited PDF plus a one-line summary of what changed.

Examples

- PDFEDIT: merge a.pdf b.pdf into combined.pdf
- PDFEDIT: rotate pages 2-4 of scan.pdf 90
- PDFEDIT: watermark report.pdf CONFIDENTIAL 0.3

Limitations

- Depends on file-reader landing first.
- AcroForm form-filling and pixel-level layout edits are not covered by pypdf.
- No OCR - a scanned page stays a scanned page.

12. translate

TRANSLATE[from>to]: | Pure LLM shim over the existing chat model | effort: ~0.5 day

Description. Strict translation that preserves formatting and adds no commentary, for any language pair the active model supports, with strong Indic coverage.

Purpose. Translation is core to a multi-Indian-language user base and is a pure prompt shim - no new runtime at all. A strict system prompt eliminates the chatty preamble that general answers produce.

When to use

- Translating text between any languages, Indic pairs included.
- Auto-detect mode when the source language is unknown.
- Bulk translation of a passage inside a document being produced.

How it works in Manthan. TRANSLATE[en>hi]: <text> (or TRANSLATE: <text> for auto-detect) calls the chat completion endpoint with a pinned system prompt: 'You are a translator. Output ONLY the translated text, preserving formatting. No commentary.' The handler renders a copy-card with original and translation side by side and offers a FILE[txt] download.

Output. A two-pane copy card (original, translation) with a copy button and a plain-text download.

Examples

- TRANSLATE[en>hi]: The report is due Friday.
- TRANSLATE: Bonjour le monde

Limitations

- Quality tracks the active model; users can switch models for better Indic output.

- Chat renders Devanagari and other scripts natively; PDF export of Indic text needs a registered font in `make_files`.

13. humanizer

HUMANIZE: | Pure LLM shim; rubric from Wikipedia's signs-of-AI-writing guide | effort: ~0.5 day

Description. Rewrites text to remove recognizable AI-writing patterns while keeping meaning and adding genuine voice.

Purpose. Users paste AI-drafted text all day and want it to not sound drafted. This is one of the highest-frequency requests and costs nothing but a well-pinned prompt.

When to use

- Text needs to sound like a person wrote it: emails, essays, posts, bios.
- Removing em-dash pileups, 'delve', rule-of-three lists and hedging boilerplate.
- Tightening tone while preserving the author's meaning.

How it works in Manthan. HUMANIZE: `<text>` runs a two-pass shim: first a diagnostic pass listing the detected AI-writing patterns with references, then a rewrite pass under instructions to add genuine voice, vary rhythm and keep all facts intact. The handler returns the original, the humanized version, and a bullet diff of what changed.

Output. A three-part card: original, humanized rewrite, and a short 'changes made' list.

Examples

- HUMANIZE: `<pasted cover letter>`
- HUMANIZE: the second paragraph above

Limitations

- Opinionated by design; the diff lets the user verify meaning survived.

14. audit

AUDIT[`type=schema|dataset|code|tos`]: | Pure LLM call; dataset and code variants execute real checks in the Pyodide sandbox first | effort: ~1 day

Description. Structured audit reports for data schemas, datasets, code and terms-of-service documents, with severity ratings.

Purpose. Audits are where a pinned output structure beats freeform prose. Borrowing the strongest QA-gate pattern from the skill catalogs: findings, severity, fixes, references - every time, in the same shape.

When to use

- Schema review: fields, types, keys, normalization.
- Dataset quality: missing values, outliers, type drift, duplicates.
- Code safety review of a pasted snippet.
- Terms-of-service clause scan for red flags.

How it works in Manthan. AUDIT[type=dataset]: <CSV text or an attached file> first runs deterministic checks in Pyodide (missing-value counts, duplicate rows, type inference, range outliers) and passes the measured facts to the model, which writes the report card: Findings, Severity (P0/P1/P2), Suggested fixes, References. The schema, code and TOS variants are prompt-only with the same card.

Output. A structured report card with four fixed sections and a severity summary line.

Examples

- AUDIT[type=dataset]: <pasted CSV>
- AUDIT[type=tos]: <pasted clauses>

Limitations

- The model does not fix anything by itself; the card lists fixes for the user to apply.

15. cite-style

CITE[apa|mla|chicago|ieee|harvard]: | Pure prompt with a target-style block | effort: ~0.5 day

Description. Converts citations between styles and validates that required fields are present.

Purpose. Students and researchers need format-exact citations, not approximately-remembered ones. This is a pure prompt skill with a fixed output shape.

When to use

- Converting a bibliography between APA, MLA, Chicago, IEEE and Harvard.
- Formatting a raw reference (title, authors, venue, year, DOI) into a style.
- Checking a citation for missing required fields.

How it works in Manthan. CITE[apa]: <reference text> applies the target style's block from the skill prompt and returns the formatted citation, plus a validation note listing any missing fields (DOI, page numbers, access date) the style expects. Batch mode converts a list one per line.

Output. The formatted citation as a copy-card, plus a short validation note.

Examples

- CITE[ieee]: Deep learning for weather forecasting, Nature 2023, doi:10.xxxx
- CITE[mla]: <pasted reference>

Limitations

- Validation catches structural omissions, not factual errors like a wrong year.

16. memory

MEMORY: (system-prompt block reading USER_PROFILE) | localStorage - the same store that already persists custom skills | effort: ~1 day

Description. A persistent per-user profile: durable facts, preferences and project context carried across sessions, readable in Settings, fully skipped in Incognito mode.

Purpose. Users re-state their name, stack, tone preferences and project context in every chat. memory removes that friction using the exact persistence Manthan already has for custom skills - no backend table needed at the

browser-local scale.

When to use

- The user states a stable fact: role, tech stack, ongoing projects, tone/format preferences.
- Any reply that would be better for knowing a previously stated fact.
- Importing a memory export from another assistant (JSON), a differentiator borrowed from Claude's import-memory.

How it works in Manthan. A bounded markdown profile (under ~2KB, headings: Profile, Preferences, Projects) lives in localStorage. At session start it is injected as a context block. The model writes with MEMORY:+ <fact> and prunes with MEMORY:- <fact>. Settings shows a viewer/editor with delete-everything. Incognito sessions neither read nor write it, and an export/import JSON pair makes it portable.

Output. Silent context injection; a visible 'remembered' note when a fact is saved; a Settings panel for review and deletion.

Examples

- MEMORY:+ I prefer concise answers with metric units
- MEMORY:- the old project name

Limitations

- Browser-local: cleared with site data unless exported - so export is prominent.
- Never store payment details, government IDs, passwords or health specifics; the write rules forbid them outright.

17. deep-research

(mode toggle, not a directive) | Orchestration over existing SEARCH + READ + Agent Swarm | effort: ~1.5 days

Description. A research mode that plans, executes and synthesizes a multi-round, multi-source investigation into a formally cited report.

Purpose. Manthan already owns every ingredient - 23 search engines, full-page reading, and a 2-10 worker swarm. What it lacks is the discipline that turns those ingredients into defensible output: forced planning, source verification, contradiction resolution and a cited final artifact. This is an orchestration pattern, not new infrastructure.

When to use

- Open-ended market, technical, competitor or policy questions that need sources.
- Any claim that must be defensible with citations.
- Multi-facet questions where one search round is clearly not enough.

How it works in Manthan. Toggling Deep Research inserts a rigid workflow into the system prompt: (1) restate and clarify the question; (2) write a research plan with sub-questions; (3) run multi-round SEARCH across engine categories, READ the best candidates, and record sources with reliability notes; (4) resolve contradictions explicitly instead of silently averaging; (5) synthesize with the Agent Swarm and emit a structured report with inline citations and a source list, optionally as FILE[md] or FILE[pdf].

Output. A formally structured report: executive answer, findings by sub-question, contradictions found and how they were resolved, and a ranked source list.

Examples

- Deep research: EV charging infrastructure policy in India 2024-2026
- Deep research: compare Pyodide vs WebAssembly runtimes for in-browser pandas

Limitations

- Consumes many search rounds and tokens; a single casual question should not trigger it (the model judges, and a hint word can force it).
- Adds latency - minutes, not seconds, for the full pipeline.

18. reminders

REMIND: | Notification API + setInterval; schedule list in localStorage | effort: ~0.5 day

Description. Reminders and recurring prompts that fire while the app is open, and surface honestly on next open if they were missed.

Purpose. The no-backend version of scheduled tasks: no push server, no service-worker guarantees - but a truthful implementation of what the browser can do, instead of pretending.

When to use

- 'Remind me in 2 hours to check the oven.'
- Recurring while-open nudges: hydration, a focus timer.
- One-shot follow-ups tied to a conversation.

How it works in Manthan. REMIND: [in 2h | at 18:30 | every 30m] <text> stores the schedule in localStorage. While the PWA tab (or its service worker) is alive, a timer fires a Notification with the text. On every app open, due-but-undelivered reminders surface as a dismissible stack at the top of the chat.

Output. A confirmation card ('Set: 18:30 today - check oven'), then a browser notification and an in-chat surfacing when due.

Examples

- REMIND: in 45m stand up and stretch
- REMIND: at 09:00 daily brief me on tech news

Limitations

- The honest limitation, stated in the UI: nothing fires when the tab is closed - there is no server to deliver it.
- Recurring items repeat only during open sessions; missed ones collapse into the next-open stack.

19. summarize

SUMMARIZE: | Chains the existing FETCH/READ directives; YouTube transcripts via the Piped API already in use | effort: ~0.5 day

Description. Summarizes URLs, YouTube videos, podcasts, articles, pasted text and attached PDFs into a consistent digest format.

Purpose. Summarization is the highest-frequency research task and is nearly free: it is a prompt pinned over fetching Manthan can already do.

When to use

- 'Summarize this article/link/video.'
- Condensing a long conversation or attached document.
- Building digests consumed by other skills (deep-research, heartbeat).

How it works in Manthan. SUMMARIZE: <url or file or text> resolves the source first - FETCH/READ for pages, the Piped API for YouTube transcripts, file-reader for PDFs - then applies a fixed digest format: one-line TL;DR, key points (3-7 bullets), notable quotes, and source links.

Output. A digest card: TL;DR, bullets, quotes, sources.

Examples

- SUMMARIZE: <https://long-article.example.com/post>
- SUMMARIZE: the PDF I attached

Limitations

- Videos without captions cannot be summarized (no transcription of arbitrary audio without keys - Whisper audio path is unchanged and separate).

20. tdd-mentor

MENTOR: / TDD: | Prompt skill executing inside the existing runpython/runjs sandbox | effort: ~0.5 day

Description. A test-driven development coach: writes the failing test first, implements against it, runs both in the sandbox, and iterates on failures.

Purpose. Manthan's code interpreter already runs code and returns output - the missing piece is the workflow discipline. tdd-mentor pins the red-green-refactor loop as instructions rather than infrastructure.

When to use

- 'Write a function for X' where correctness matters.
- Reviewing or fixing a pasted snippet.
- Teaching mode: explaining each step of the loop as it runs.

How it works in Manthan. TDD: <task> forces the order: (1) write a minimal failing test in a fenced runpython/runjs block; (2) run it, show the failure; (3) implement; (4) run again, show the pass; (5) refactor with the test as the guardrail. MENTOR: is the Socratic variant that guides the user to write the code themselves, one step per turn.

Output. Alternating code blocks and test output in the chat, ending with a passing suite and a short lessons-learned note.

Examples

- TDD: a Roman numeral converter in Python
- MENTOR: teach me recursion with exercises

Limitations

- Sandbox limits apply (no network in Pyodide, no native extensions).

21. design-polish

DESIGN[theme=]: | System-prompt branch biasing live_preview output; Mimo's critique/typeset/polish vocabulary as prompt guidance | effort: ~0.5 day

Description. Raises anything rendered through Live Preview to designer grade: typography, spacing rhythm, intentional color and restrained motion.

Purpose. Manthan can already render HTML/React live; what separates its output from a designer's is a set of tastes the model was never told to apply. design-polish injects those tastes, plus a self-review pass.

When to use

- Any Live Preview build: tools, dashboards, landing pages, prototypes.
- Applying a named theme: DESIGN[theme=brutalist], [theme=editorial], [theme=minimal].
- Critique mode on an existing preview.

How it works in Manthan. When the skill is on, a system-prompt branch requires: a deliberate type scale with a display face choice, consistent 4/8px spacing rhythm, an intentional palette (not default blues), accessible contrast, and restrained motion. After rendering, the model critiques its own output against the same checklist and fixes violations before finishing. DESIGN[theme=...] selects a named preset.

Output. Live Preview output that visibly follows the checklist, plus a short 'design notes' line describing the choices made.

Examples

- DESIGN[theme=editorial]: build a personal blog homepage
- DESIGN: redo the KPI dashboard with better hierarchy

Limitations

- Taste is prompt-deep, not pixel-enforced; the checklist catches the worst defaults, not every flaw.

22. self-improvement

(automatic, no user directive) | Markdown log in localStorage alongside memory | effort: ~0.5 day

Description. Records the user's corrections of the model and injects the recent ones as context, so the same mistake is not repeated next session.

Purpose. Every correction a user types ('no, I said metric units') is free training signal that currently evaporates. A one-line append turns it into durable behavior - the browser-safe version of Mimo's self-improvement log.

When to use

- The user corrects the model's assumption, format preference or factual slip.
- The model notices it repeated an earlier mistake in the same session.

How it works in Manthan. When a correction is detected (explicit 'no, actually' patterns, or a user re-stating a preference the reply violated), the model appends one line to a localStorage log: date, the wrong behavior, the

correction. The five most recent lines ride along with the memory context block each session. The log is viewable and deletable in Settings, and never written in Incognito.

Output. A small 'noted' toast on correction; better-formed answers in later sessions.

Examples

- (no direct invocation - fires on corrections)

Limitations

- Heuristic detection only flags clear corrections; ambiguous ones are skipped rather than guessed.

23. heartbeat

(periodic hidden prompt + localStorage due-list) | setInterval while open; due-check on app open | effort: ~1 day

Description. A periodic check-in that composes the daily brief, surfaces due reminders and prunes stale memory - deciding on its own whether anything is worth saying.

Purpose. Mimo's HEARTBEAT.md pattern, browser-honest: while the app is open, or on the next open if something came due, the assistant gets a hidden turn to act on accumulated state instead of waiting to be asked.

When to use

- A daily brief the user asked for (news, weather, reminders digest).
- Surfacing due reminders on open.
- Periodic memory hygiene: compress, deduplicate, prune stale facts.

How it works in Manthan. A timer (default: every few hours while open, plus a due-check at open) fires a hidden prompt with the memory block, reminder list and any subscribed brief topics. The model decides: compose a brief, remind quietly, or stay silent. Output appears as a normal message with a small heartbeat badge.

Subscription is per-topic via natural language and revocable the same way.

Output. A composed brief or reminder message when warranted; silence otherwise.

Examples

- (fires on schedule; no user invocation beyond subscribing to briefs)

Limitations

- Same honest limit as reminders: nothing happens while the app is fully closed.
- Brief quality depends on the enabled search engines at fire time.

Part 2 - Prompt-only library skills (MAKESKILL ports)

Twenty pure-instruction skills, grouped by category. Each entry is a complete skill body: the description (used as the trigger frontmatter), the working method, and the output contract. Ported into the existing MAKESKILL format they cost nothing at runtime - toggled prompt text, exactly like a custom skill a user could write, only shipped built-in and maintained.

Category: Writing

24. copywriting

Source: Kimi

Description. Marketing and conversion copy for pages: home, landing, pricing, features, about, product.

A prompt-only skill that writes to a conversion brief. Workflow: identify the page type and audience, pick a value proposition angle, write headline variants plus body sections, and mark each claim that needs a factual source before shipping. Triggers on requests for landing-page copy, product descriptions, ad text and hero sections. Output: structured copy blocks (headline, subhead, body, CTA) with alternates for A/B testing.

25. copy-editor

Source: Kimi

Description. Seven-pass professional copy editing of any pasted text.

Runs a fixed sequence of editorial passes - correctness, clarity, structure, flow, tone, consistency and mechanics - and reports what each pass changed. Triggers on 'edit this', 'proofread', or a paste plus a quality request. Output: the edited text plus a per-pass change summary, so the user can accept or revert layer by layer.

26. brand-naming-lab

Source: Kimi

Description. Generates and stress-tests brand, product and feature names.

Expands a naming brief into candidate names across strategies (descriptive, evocative, coined, compound), then screens each candidate for pronounceability, domain-squatting likelihood, unintended meanings across Indian languages and English, and trademark-shaped conflicts. Output: a shortlist with rationale and per-name risk notes.

27. cv-tailor

Source: Kimi

Description. Tailors a resume and cover letter to a specific job description.

Reads the pasted CV and the job posting, maps experience to requirements, rewrites bullets in accomplishment-verb form with quantified outcomes, and flags honest gaps rather than inflating them. Output: a tailored CV plus a matching cover letter and a requirements-coverage table.

Category: Education

28. academic-paper-reviewer

Source: Kimi

Description. Simulated peer review across originality, methodology, results and writing, with Major/Minor revision verdicts.

Applies a referee's structure to a pasted paper or draft section: summary, originality assessment, methodology critique (sample size, controls, statistics), results interpretation, and writing quality. Ends with a verdict - Accept, Minor Revision, Major Revision - and a numbered list of required changes. Pairs naturally with bloom-quiz-maker for study material.

29. bloom-quiz-maker

Source: Kimi

Description. Generates quizzes and mock exams mapped to Bloom's taxonomy levels, with answer keys.

Builds assessments ordered by cognitive level: Remember, Understand, Apply, Analyze, Evaluate, Create - mixing MCQ, short answer and scenario questions with a marking scheme. Designed to pair with Manthan's existing MP Board examiner persona: past syllabus or chapter text in, board-pattern mock exam out. Output: exam paper, answer key, and a topic-coverage map.

30. code-mentor

Source: Kimi

Description. Interactive tutoring: explains, exercises and checks understanding one step per turn.

A Socratic tutor for programming: diagnoses the learner's level with one question, explains one concept per turn at that level, sets a small exercise, waits, then reviews the attempt with targeted feedback. Runs code samples through the existing sandbox when an exercise needs execution. Output: one focused teaching turn at a time - never a wall of lecture.

Category: Planning

31. plan-mode

Source: MiniMax

Description. Talk before you build: present a plan and options, get approval, then execute.

When a task is multi-step or produces an artifact, plan-mode requires the model to first emit a short plan - steps, tools, output format, open choices - and wait for the user's go-ahead before touching a single tool. Triggers on build/make/generate verbs for non-trivial tasks. Output: a numbered plan card with approval buttons; execution only proceeds on approval. This is the single cheapest reliability upgrade in the pack.

32. cross-examine

Source: Kimi

Description. Interrogates an idea or plan before it is built, playing skeptic on purpose.

Given a plan, spec or argument, the skill lists the strongest objections: unstated assumptions, failure modes, missing requirements and scale/effort misestimates - then asks the user only the questions whose answers would change the design. Output: an objection list ranked by severity, plus the decisive questions.

33. deep-probe

Source: Kimi

Description. Clarifying-question discipline for ambiguous requests.

When a request is ambiguous enough that guessing would waste a full build, deep-probe generates a small set of targeted questions - never more than five - each with suggested defaults so the user can answer in one tap.

Output: a question card with defaults; the user's answers are folded into the task before execution.

Category: Design

34. critique

Source: Mimo

Description. Evaluates a UI from a UX perspective: hierarchy, architecture, emotional resonance, AI-slop detection.

Reviews an existing Live Preview build or pasted design against a UX rubric: visual hierarchy, information architecture, emotional resonance and - notably - detection of generic AI-design patterns (default blues, centered hero, three-card rows). Output: a prioritized findings list with concrete fixes, each mapped to what it improves.

35. typeset

Source: Mimo

Description. Typography repair: font choice, hierarchy, sizing, weight consistency and readability.

A focused pass over text-heavy output: choose a deliberate type pairing, fix the scale (modular ratios over ad-hoc sizes), align weights to meaning (not decoration), set measure and leading for readability. Applies to Live Preview HTML and to FILE[docx/pdf] templates alike. Output: the corrected artifact plus a short note on the typographic decisions.

36. arrange

Source: Mimo

Description. Layout, spacing and visual rhythm repair.

Fixes monotonous grids and weak hierarchy: recomposes layout with intentional grouping, alignment and whitespace rhythm, and varies density so the page reads in the intended order. Works on Live Preview output and deck/slide layouts. Output: restructured layout with before/after notes.

37. polish

Source: Mimo

Description. The final pre-ship quality pass: alignment, spacing, states, copy and detail.

A meticulous checklist sweep executed just before delivering any visual artifact: pixel alignment, consistent spacing, hover/focus/active states, empty and error states, microcopy, and small details (favicon, focus rings, truncation). Output: the polished artifact plus a 'checked N items, fixed M' line.

38. harden

Source: Mimo

Description. Strengthens interfaces against edge cases, errors, internationalization and real usage.

Reviews a build for what breaks under reality: long strings and translations, empty and slow states, rapid clicks, offline, RTL scripts, unusual data. For each, applies or recommends the guard. In a browser app this includes Devanagari and other Indic script rendering, number-format variants, and touch targets. Output: a hardened build with a list of the cases now covered.

39. delight

Source: Mimo

Description. Adds moments of joy: micro-interactions, personality and easter eggs that serve usability.

Where an interface is functionally done but flat, delight adds purposeful touches - motion that confirms actions, empty-state illustrations, tasteful sound-off personality - always tied to feedback rather than decoration for its own sake. Output: the upgraded build with a list of interactions added and the usability reason each serves.

40. onboard

Source: Mimo

Description. Designs onboarding flows, empty states and first-run experiences.

Plans the path from first open to first value: progressive disclosure of features, empty states that teach, and a first-run flow that gets the user to a win before asking for configuration. Output: flow outline plus implementation in the Live Preview build.

Category: Data

41. dataset-quality-audit

Source: Kimi

Description. Runs quality checks on tabular data across twelve dimensions and outputs a report card.

Given a CSV (pasted or attached via file-reader), executes deterministic checks in the Pyodide sandbox - completeness, uniqueness, type validity, range outliers, cross-field consistency, temporal ordering and more - then composes the findings into the audit skill's report-card format with severity ratings. Output: a quality report with per-dimension scores and the offending rows listed.

42. auto-stat-test

Source: Kimi

Description. Chooses and runs the correct statistical test automatically.

Inspects the data's shape (groups, pairing, distribution, sample sizes), states which test applies and why (t-test, Mann-Whitney, chi-square, ANOVA, correlation), runs it in Pyodide with scipy, and interprets the output in plain language with effect size and caveats. Output: the chosen test with rationale, results, and a one-paragraph interpretation.

Category: Wellness

43. health

Source: Meta AI

Description. Loads before any health question: evidence-based answers with safety flags and crisis resources.

A pure prompt skill with zero runtime: when a question touches health, wellness or medicine, the skill's instructions load first - answer from established evidence, distinguish consensus from emerging findings, flag red-flag symptoms that warrant a doctor, never diagnose, and surface crisis resources appropriate to the user's locale when a question suggests distress. Output: a clear answer with a calm 'see a professional' line where warranted - never preached, never withheld.

Part 3 - Recommended order of implementation

- chart-studio - the biggest visible gap, entirely client-side.
- Built-in prompt-skill library - the 20 MAKESKILL ports; near-zero runtime cost.
- memory - the differentiator; reuses the localStorage pattern custom skills already use.
- deep-research - orchestration only, over what exists.
- speak, convert, weather, image-search - quick directive wins (half a day each).
- file-reader - prerequisite for pdf-edit and the dataset skills.
- image-gen, then gif-maker - the media pair on the keyless provider.
- places, stock-snapshot, summarize, translate, humanizer, audit, cite-style, tdd-mentor, design-polish - the shims and search cards.
- reminders, heartbeat, self-improvement - the while-open automation trio, with their honest limits stated in the UI.

What is deliberately not in this pack

Excluded by the no-backend / no-OAuth / no-OCR constraints, and listed so the omissions read as decisions rather than gaps: email and calendar (Gmail/Outlook - OAuth clients), headless browser automation (server Chromium), video generation (paid hosted models), the Meta Ads suite and post-coach/rival-watch (Meta OAuth), server-side finance libraries (akshare/Tushare), coding-agent harnesses (local CLIs), R2 upload (needs a presigned-URL endpoint), voice cloning (server models), and OCR (removed per review).

Derived from the Skill Gap Analysis, the Skill Expansion Plan, the Skills Assessment, the Feature Suggestions document, and the GLM / GPT consolidation analyses. Compiled 25 September 2026 for Manthan Agent v7.9.12.